How-To Library

Informatica

Ingest bulk data from multiple Kafka topics into Hive or HDFS using streaming mapping

© Copyright Informatica LLC 2020, 2021. Informatica, the Informatica logo, and Informatica Intelligent Cloud Services are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

You can use Data Engineering Streaming mappings to capture data from multiple data sources and ingest it into a target. This article describes how you can ingest CDC data from multiple Kafka topics into Data Engineering systems in a single mapping and publish the data to either HDFS or Hive partitioned tables.

Supported Versions

- Data Engineering Streaming 10.4.1

Table of Contents

Change Data Capture Overview. 2
Change Data Capture Use Cases. 3
Bulk Data Ingestion Solution. 4
Overview. 4
Prerequisites for Bulk Data Ingestion into HDFS. 5
Schema Location for a Dynamic Schema Strategy. 6
Prerequisites for Bulk Data Ingestion into Hive Partitioned Tables. 6
Create a Streaming Mapping. 8

Change Data Capture Overview

Data with transient value comes from multiple sources such as changes from transactional systems, log files, machine logs, IoT devices, and social media. To ensure that you do not miss the opportunities in transient insights, it is important to have a process to rapidly capture data changes and updates from transactional data sources. Change Data Capture (CDC) is a process of capturing changes at the data source and then apply them throughout the enterprise. In relational transactional databases, you can use the CDC technology to capture the changes in a system as they occur and propagate the changes into analytical systems for real-time processing and analytics.

Change Data Capture continuously synchronizes and loads incremental data from multiple systems. You can use CDC to dynamically modify the data stream to accommodate schema changes for the incoming data. Database and data warehousing environments require CDC solutions because capturing and preserving the state of data across a period of time is a core function of a database and data warehouse. Use CDC to continuously capture real-time changes in production data that the source has registered during a given time period and then ingest the data into target systems such as a database or data warehouses.

Change Data Capture has the following three fundamental advantages over batch replication:

- CDC enables faster and more accurate decisions based on the most current data. For example, inserting data about database transactions to streaming analytics applications.
- CDC minimizes disruptions to production workloads.
- CDC reduces the cost of transferring data over the wide area network by sending only incremental changes.

These advantages enable IT organizations to meet the real-time, efficiency, scalability, and low-production impact requirements of a modern data architecture.

There are multiple methods and technologies to achieve CDC, and each has its merit depending on the use case. Three common methods are timestamps, triggers, and log based CDC.

Timestamps. Timestamps in a dedicated source table column can record the time of the most recent update. This method is the most widely used CDC technique for extracting incremental data. This approach retrieves rows that changed since the data was last extracted.

Triggers. Triggers log transaction events in an additional shadow table that you can play back to copy those events to the target on a regular basis. Shadow tables might store the entire row to keep track of every single column change, or they might store only the primary key and operation type.

Log based CDC. Log readers identify new transactions by scanning changes in transaction log files that already exist for backup and recovery purposes. Transactional databases store all changes in a transaction log that the database can use to recover in the event of a crash. With log based CDC, you can read new database transactions such as, insert, update, and delete from source database transactions. Changes are captured without making application level changes and without having to scan operational tables, both of which add additional workload and reduce source systems' performance. Log based CDC is the most preferred, fastest, and least disruptive CDC method because it requires no additional modifications to existing databases or applications.

Change Data Capture Use Cases

Change Data Capture (CDC) is used for various use cases such as synchronization of databases, real-time streaming analytics, ingestion into data lake and data warehouses.

Real time data synchronization or replication. Organizations use databases for operational reporting. They can employ batch and CDC replication to copy the necessary records to reporting databases, thereby offloading the queries and analytics workload from production. Reference architectures can become complex in large enterprises, where different teams are working on different sets of technologies and each has its requirements. When you apply CDC methodology, you need to modify the application processing mechanism to create a unified solution across all systems. As your data is loaded into different types of discrete systems, you may need to apply in-memory, filtration, and transformation, and other types of operators to the data. Your data is processed and pushed down to the system that is the target for your CDC-based needs. You may want to process your data in batches. This is done mostly for the first time you load your data from source system to your target system. Once you have your initial data loaded, you may want to process the incremental changes in real time.

Real-time streaming analytics. CDC can propagate data to message queues to help analyze streaming log data for any kind of issue. In this case, data is pushed to a messaging queue like Apache Kafka or Amazon Kinesis as a target for reading and processing data. CDC can also be leveraged for data migration from on-premises to cloud, advanced analytics, and machine learning use cases by analyzing data to generate real-time insights and data modernization initiatives.

Real-time data lake and data warehouse ingestion. In a modern data architecture, organizations can continuously ingest CDC data into a data lake or data warehouse in the form of an automated data pipeline. CDC helps avoid some of the bottlenecks you might encounter when provisioning large amounts of data from legacy data stores to the new data lake or data warehouse as the data only needs to be loaded once. Thereafter, CDC just provisions the changes to the data lake or data warehouse. In fact, many organizations have used data lake over ETL platforms, as the data lake or data warehouse environment tends to be less expensive. The most difficult part of the data lake or data warehouse is maintaining it with current data. CDC can be extremely helpful there. It can help you save computing and network costs, especially in the case of cloud targets, as you are not moving terabyte of data unnecessarily across your network. You can focus on the change in the data.

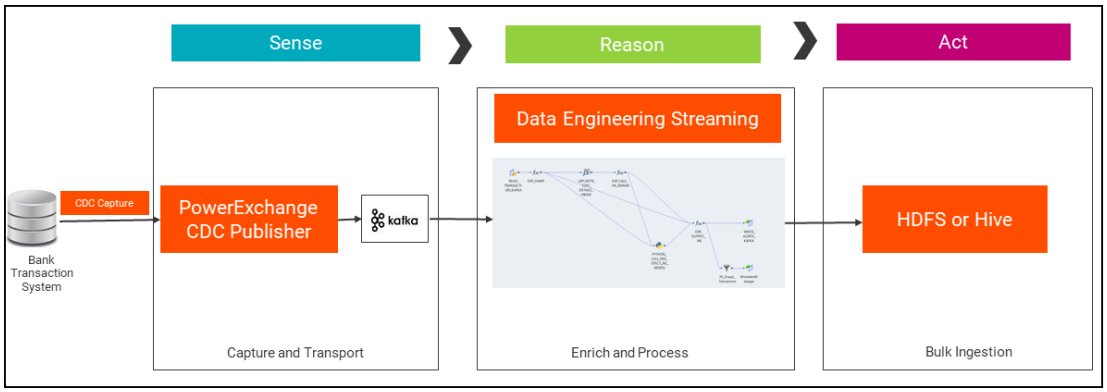
This article focusses on the real-time data lake and data warehouse ingestion use case. This article provides solution about how you can use a single data engineering streaming mapping to handle bulk transactional data ingestion into a data warehouse. You can then process the ingested data for downstream analytics.

Bulk Data Ingestion Solution

Scenario

Change Data Capture efficiently identifies and captures data that has been added to, updated in, or removed from, relational tables and makes this change data available for use by downstream applications. In a banking scenario, a bank needs to capture large amount of change data continuously as new database transactions occur, process, and then upload the changed data to the destination database systems or data warehouse. Informatica Data Engineering Streaming mapping can continuously capture the changed data by leveraging open source technologies such as Apache Kafka, enrich and process the data as required, and then ingest the data to data lakes or data warehouses for further processing.

The following image shows the reference architecture for bulk data ingestion using a streaming mapping:



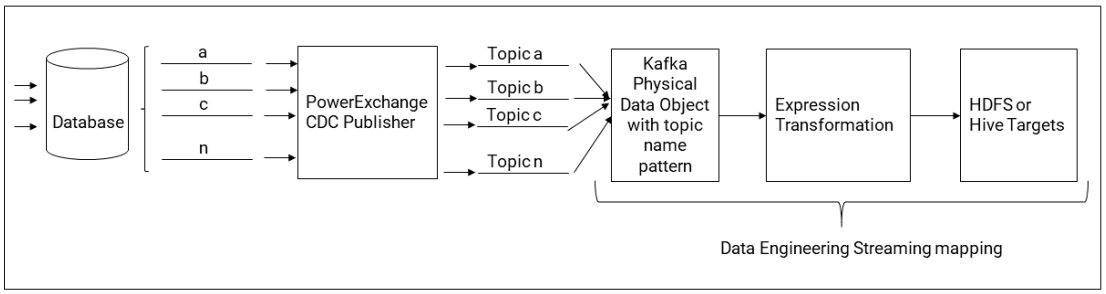
In a transactional system, the PowerExchange CDCPublisher gathers incoming data that has been modified in the source and publishes the data to Apache Kafka. Apache Kafka creates individual topics for the corresponding source tables. Data Engineerng Streaming mapping then uses these topics and loads them into the HDFS or Hive tables.

Overview

Use Data Engineering Streaming mapping to stream change data capture from multiple Kafka sources and write to target messaging systems such as HDFS and Hive partitioned tables. After you publish the change data to the target messaging system, consumer applications can read the data as required. However, the real challenge occurs when you must capture the change data for hundreds of tables and write to target tables for further analytics.

For example, if you have 300 Kafka topics, you must create 300 different mappings to read data from each Kafka topic. That is, to read a single topic, you create one dedicated mapping. This results in creating one YARN application for one topic which in turn leads to huge resource consumption on the cluster nodes. To solve this problem, you can create a single Data Engineering Streaming mapping to read from multiple Kafka topics using the Kafka topic patterns property. You can then publish the data to separate targets such as HDFS files or Hive partitioned tables. Each source table corresponds to a separate target HDFS file created using *FileName* port.

See the following image for a graphical representation of the overall process of bulk data ingestion:



Real-time data in high volume enters multiple database tables. This data can either be new data or modified data. PowerExchange CDC Publisher gathers this recently changed or modified data in the source. It creates individual Kafka topic for the corresponding source tables. The topic names that are input to the mapping must have a common pattern in them so that the mapping source can read from all the topics at the same time. This topic name pattern matches to the pattern configured in the Kafka physical data object in the mapping. All topics that has the messages aligned to the defined patterns are read, processed, and then written to the target. In the target, each topic name corresponds to a file name port. Thus, target files are created corresponding to each topic name.

Prerequisites for Bulk Data Ingestion into HDFS

You can use a streaming mapping with a Kafka or a Confluent Kafka source to ingest bulk data into HDFS target.

Complete the following prerequisites before you ingest bulk data into HDFS:

- Generate an Avro schema file for a respective Kafka topic and name the schema file as `reader.avsc`.
- Save the schema file directory in the external location for HDFS, Amazon S3, and Microsoft Azure Data Lake Storage Gen2 with the following format:

  ```
  /<Schema location>/<Topic name>/
  ```
- Ensure that the complex file data object target is of binary file format.
- Connect the FileName port from upstream transformation and with a non-empty expression.
- Disable column projection in the complex file data object target.

Schema Registry

To publish CDC data to a HDFS target, you can either use the same schema for both read and write operations or use different schemas for both read and write operations.

When you use a valid Schema Registry and a different schema for the target, ensure to complete the following prerequisites:

- Upload the Avro schema file to the external location of the target.
- Name the Avro schema file as `writer.avsc`.
- Save the `writer.avsc` file in the following location:

  ```
  <SchemaLocation>/<TopicName>/writer.avsc
  ```

When you use the same schema for both read and write operations:

- For Confluent Kafka sources if you select Schema Registry, then schema is fetched from the registry. If you select External Location, then schema is fetched from the External Location for HDFS, Amazon S3, or Microsoft Azure Data Lake Storage Gen2.
- For Apache Kafka sources schema is fetched from the External Location for HDFS, Amazon S3, or Microsoft Azure Data Lake Storage Gen2.

When you use different schemas for both read and write operations:

- For Confluent Kafka sources, schema is fetched from the Schema Registry. Schema is fetched from the External Location for the target. External Location can be HDFS, Amazon S3, or Microsoft Azure Data Lake Storage Gen2.
- For Apache Kafka sources schema is fetched the External Location. Schema is fetched from the same External Location for the target.
- If Schema Location for the schema is not provided in the Complex File Data Object target, then the same source schema is used.

External Location

When you use a valid External Location, ensure to complete the following prerequisites:

- Upload the Avro schema file to the external location of the target.
- Name the Avro schema file as `reader.avsc` for the source and `writer.avsc` for the target.
- Save the `reader.avsc` and `writer.avsc` file in the following location:

```
<Schema location>/<Topic name>/reader.avsc

<Schema location>/<Topic name>/writer.avsc
```

Schema Location for a Dynamic Schema Strategy

The following table lists the schema location for a dynamic schema strategy:

Source Schema Location	Target Schema Location	Resultant Source Schema	Resultant Target Schema
Schema registry	Empty	Schema registry	Schema registry (same as source)
Schema registry	External location	Schema registry	External location
External location	Empty	External location	External location (same as source)
External location	External location	External location	External location
Confluent Kafka Data Object (Schema Registry URL is set in the connection) and External Location	Empty	External location	External location (same as source)
Confluent Kafka Data Object (Schema Registry URL is set in the connection) and External Location	External location	External location	External location
None	External location	No dynamic schema strategy	No dynamic schema strategy
None	Empty	No dynamic schema strategy	No dynamic schema strategy

Prerequisites for Bulk Data Ingestion into Hive Partitioned Tables

You can use a streaming mapping with a Kafka source to ingest bulk data into Hive partitioned tables.

Complete the following prerequisites before you ingest bulk data into Hive partitioned table:

- Generate an Avro schema file for a respective Kafka topic and name the schema file as `reader.avsc`.

  The input data from the source must be in Avro encoded binary format.
- Save the schema file directory in the external location for HDFS, Amazon S3, and Microsoft Azure Data Lake Storage Gen2 with the following format:

  ```
  /<Schema location>/<Topic name>/
  ```
- Create Hive table with same as topic name. The Hive tables must exist before starting the mapping. Ensure that the column names in the Hive table are in lower case.
- In the cluster where you want to run the mapping, save the schema file directory in the following location:

  ```
  /Parent directory/<database name>.<topic name>
  ```

  This location is on the same file system as **State Store Connection**.
- Create a streaming mapping with Kafka source and Hive partitioned table as the target. Configure the following run-time properties of the mapping:

  ```
  infaspark.hive.impersonated.complexfile.write=true
  ```
- Add a transformation to add database name before the topic name of the incoming topic. However, if the Kafka topic name is of the pattern `database name.topic name`, you do not need to change the target schema name.
- Configure the file name port value to `database name + topic name`.
- Connect the FileName port from upstream transformation and with a non-empty expression.
- Disable column projection in the complex file data object target.

Schema Registry

The source schema is picked from Schema Registry. Create Hive partitioned table with the same name as the topic name. Ensure the with column names in the tables are same as the source schema column names.

To write CDC data to a Hive target, ensure that the target Hive table conforms to the following specific schema requirements:

- All columns in target data should have the same name and compatible data type, such as timestamp and date, as that of the source Kafka topic.
- If the Hive target table is partitioned, then the column on which the partitioning is applied must be available in the source data with the same name and compatible data type.
- If the source data has lesser number of columns than the target Hive table, then the additional columns in the Hive target table has Null as value.
- If the source data has more number of columns than the target Hive table, then the additional columns in the source schema is ignored while writing.
- Column projection on the source is not available.

External Location

When you use a valid External Location, ensure to complete the following prerequisites:

- Upload the Avro schema file to the external location of the target.
- Name the Avro schema file as `reader.avsc` for the source.
- Save the `reader.avsc` file in the following location that is on the same file system as **State Store Connection**:

  ```
  <Schema location>/<Topic name>/reader.avsc
  ```

Create a Streaming Mapping

1. Import the Kafka Data Object that is created using Kafka Topic Patterns.
2. Import any complex file data object from the HDFS connection.
3. Create a mapping and add the Kafka Data Object as read object.
4. Add an Expression transformation that connects the Kafka topic name port and the data port from source to the Expression transformation.
5. In the Expression transformation, complete the following configurations:
   - For HDFS target, map the Kafka topic name to the FileName port.
   - For Hive target, create a new port and perform the `concat(database name,topic name)` in the new port.
6. Add the complex file target as the write object.
7. For Hive target, set **Dynamic Schema Strategy** to **External Location** in Kafka Data Object.

   Add the following run-time property at mapping level:

   ```
   infaspark.hive.impersonated.complexfile.write=true
   ```

Data Object Read Operation

You can create a data object read operation and configure the data object read operation properties in the object editor.

To read the Kafka CDC data, choose whether you want to use Schema Registry or External Location to fetch the schema information.

If you choose **Schema Registry**, then the Schema Registry URL is used to fetch the schema information. This option is available only for Confluent Kafka Data Object.

If you choose **External Location**, then enter the Schema Location.

Confluent Kafka Read Operation

You can create or import a Confluent Kafka data object. When you configure a Confluent Kafka data object, you can specify the topic name or use a regular expression for the topic name pattern only when you read from Kafka.

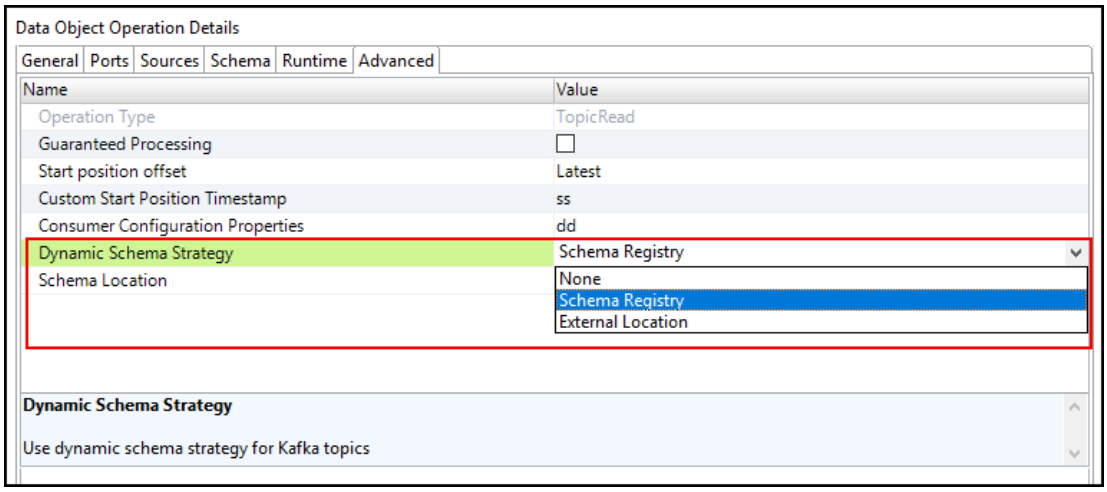
While creating a Confluent Kafka data object, specify the data format as binary. At run time, the schema is picked from the external location and the binary data gets parsed as Avro data with the help of the schema.

The dynamic schema strategy identifies the schema at run-time for ingesting streaming data into files or tables in a mapping. Only Avro schema is supported. You must disable the column projection. You can select one of the following strategies:

- None
- Schema Registry
- External Location

If you select **External Location** for the dynamic schema strategy, you must create a `reader.avsc` file that has the schema content at the schema location. Save the `reader.avsc` within the topic name path. For example: `<Schema location>/<topic name>/reader.avsc`. Then, specify only the path up to the schema location.

The following image shows the **Advanced** tab of the Confluent Kafka read operation:



Kafka Read Operation

When you configure the Kafka data object, specify the topic name that you read from or write to. You can specify the topic name or use a regular expression for the topic name pattern only when you read from Kafka.

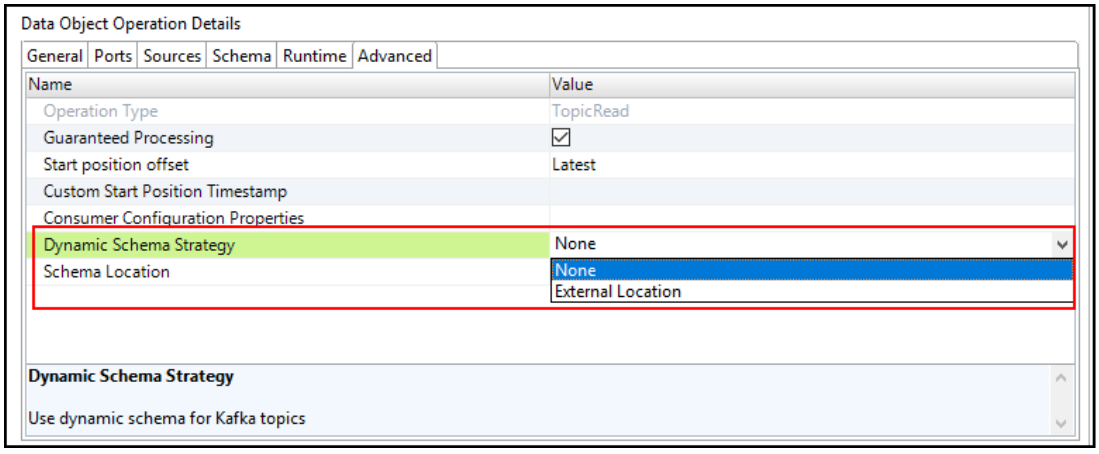
After you create a Kafka data object, create a read operation. You can use the Kafka data object read operation as a source in streaming mappings.

The dynamic schema strategy identifies the schema at run-time for ingesting streaming data into files or tables in a mapping. You can select one of the following strategies:

- None
- External Location

If you select **External Location** for the dynamic schema strategy, you must create a `reader.avsc` file that has the schema content at the schema location. Save the `reader.avsc` within the topic name path. For example: `<Schema location>/<topic name>/reader.avsc`. Then, specify only the path up to the schema location.

The following image shows the **Advanced** tab of the Kafka read operation:



Supported Transformations

You can use the following transformations:

- Java transformation
- Expression transformation
- Router transformation
- Filter transformation

Data Object Write Operation

You can create a data object write operation and configure the data object write operation properties in the object editor.

To write the incoming change data, choose whether you want to write to the same schema format or a different schema format.

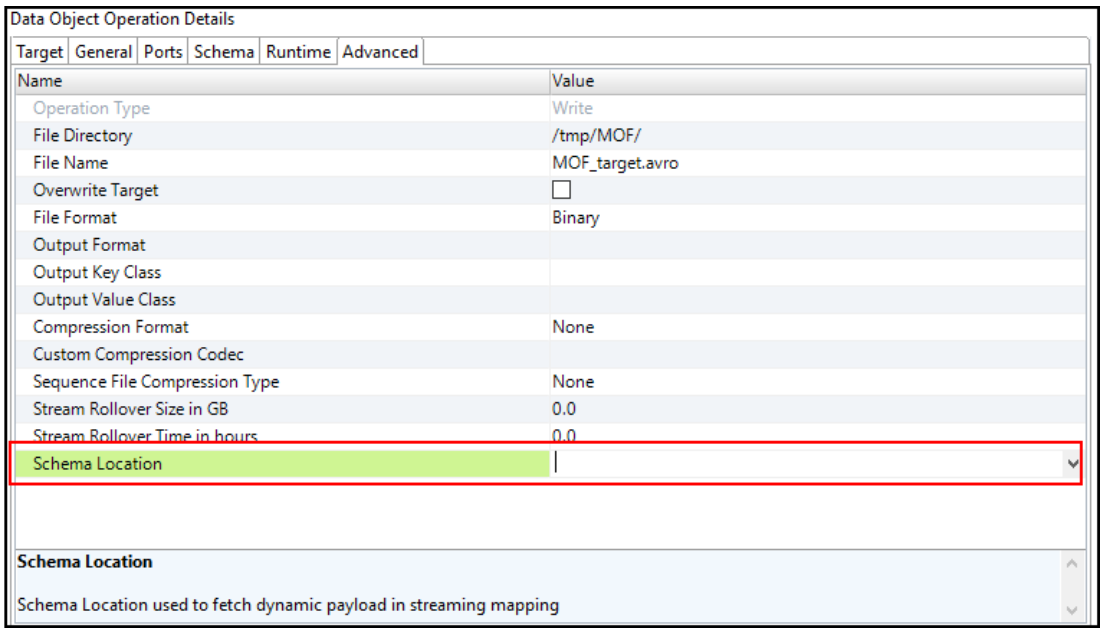
If you choose to write data to different schema format other than the source schema, then provide the **Schema Location**. If you choose to write data to same schema format, then leave the **Schema Location** field empty.

Complex File Data Objects Write Operation

Create a complex file data object with an HDFS connection to write data to HDFS binary files. When you create a data object write operation for HDFS complex files, the FileName port appears by default.

If **Schema Location** is set to a non-empty value, then create a `writer.avsc` file that has the schema content at the schema location. Save the `writer.avsc` within the topic name path. For example: `<Schema location>/<topic name>/writer.avsc`. Then, specify only the path till the schema location.

The following image shows the **Advanced** tab of the Complex File Data object write operation:



Authors

Pubali Dekaphukan

Aman Agarwal